

Arquitectura de Computadores - Resumen Clase 18

Martes 5 de mayo - AVX, punto flotante escalar/empaquetado y comparaciones

Idea central: La clase conecta la teoría de aritmética finita con la programación AVX: primero compara aritmética wraparound y saturada; luego presenta registros YMM/XMM, MXCSR, tipos de datos AVX/SSE, operaciones escalares de punto flotante, depuración en Visual Studio y operaciones empaquetadas SIMD.

1. Aritmetica wraparound vs aritmetica saturada

En aritmetica entera de precision finita, el procesador trabaja con una cantidad fija de bits. Si el resultado real no cabe, se necesita definir que hacer con el overflow o underflow.

Tipo	Comportamiento	Ejemplo mental
Wraparound	Se permite el overflow. El resultado se interpreta como aritmetica modular: al pasarse del maximo, da la vuelta dentro del rango representable.	En 16 bits con signo, $20000 + 15000$ no cabe; en wraparound puede terminar como un valor negativo.
Saturada	El resultado se limita al minimo o maximo representable. Si se pasa del maximo, devuelve el maximo; si baja del minimo, devuelve el minimo.	$20000 + 15000$ se satura en 32767. $-5000 - 30000$ se satura en -32768.

En clase se usaron ejemplos con enteros de 16 bits. El rango con signo es de -32768 a 32767. Por eso, 35000 no cabe como entero positivo de 16 bits con signo, y -35000 tampoco cabe como negativo.

Para el trabajo: Cuando veas un resultado raro despues de una suma o resta de enteros, pregunta primero si la operacion esta haciendo wraparound. Si el contexto es saturado, el resultado deberia quedarse pegado al limite del rango.

2. Ambiente de programacion AVX: YMM, XMM y banco de registros

AVX trabaja con 16 registros de 256 bits llamados YMM0 a YMM15. Los 128 bits mas bajos de cada YMM se pueden acceder como los registros XMM0 a XMM15. Esto permite usar el mismo banco de registros para operaciones escalares, vectoriales de 128 bits y vectoriales de 256 bits.

Registro	Tamano	Uso importante
XMM0-XMM15	128 bits	SSE y operaciones escalares FP. En escalar se usan solo los 32 o 64 bits bajos.
YMM0-YMM15	256 bits	AVX. Permite operar varios floats o doubles empaquetados en una sola instruccion.
MXCSR	Registro control/status	Guarda banderas de estado, excepciones y opciones de redondeo para punto flotante SIMD/AVX.

El registro MXCSR es especialmente importante porque contiene banderas de status para detectar condiciones anomalas en instrucciones de punto flotante, y bits de control para habilitar/deshabilitar excepciones o elegir opciones de redondeo.

3. Tipos de datos SSE/AVX

La misma anchura de registro puede verse como varios datos pequenos empaquetados. El significado depende de la instruccion que se use, no de un tipo fuerte como en alto nivel.

Familia	Tipo comun	Interpretacion	Cantidad por registro
SSE/XMM	m128	float de 32 bits	4 floats
SSE/XMM	m128d	double de 64 bits	2 doubles
SSE/XMM	m128i	enteros empaquetados	bytes, shorts, ints, longs, etc.
AVX/YMM	m256	float de 32 bits	8 floats
AVX/YMM	m256d	double de 64 bits	4 doubles
AVX/YMM	m256i	enteros empaquetados	mas relevante con AVX2

4. Operaciones escalares de punto flotante con XMM

Aunque AVX se asocia con procesamiento vectorial, también permite operaciones escalares. En ese caso, se usan los 32 bits bajos para precisión simple o los 64 bits bajos para double. El resto del registro puede conservar otros bits o quedar sin interés para el cálculo.

El ejemplo trabajado fue la distancia euclídea entre dos puntos en 3D:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Datos del ejemplo observado: $x_1=1.0$, $y_1=2.0$, $z_1=3.0$, $x_2=5.0$, $y_2=5.0$, $z_2=5.0$. Entonces el resultado esperado es $\sqrt{16+9+4}=\sqrt{29}$, aproximadamente 5.385164807.

```
.data
    x1 qword 1.0
    y1 qword 2.0
    z1 qword 3.0
    x2 qword 5.0
    y2 qword 5.0
    z2 qword 5.0
    Distancia qword ?

.code
main PROC
    vmovsd xmm0, x1
    vmovsd xmm3, x2
    vmovsd xmm1, y1
    vmovsd xmm4, y2
    vmovsd xmm2, z1
    vmovsd xmm5, z2

    vsubsd xmm0, xmm3, xmm0    ; x2 - x1
    vmulsd xmm0, xmm0, xmm0    ; (x2 - x1)^2

    vsubsd xmm1, xmm4, xmm1    ; y2 - y1
    vmulsd xmm1, xmm1, xmm1    ; (y2 - y1)^2

    vsubsd xmm2, xmm5, xmm2    ; z2 - z1
    vmulsd xmm2, xmm2, xmm2    ; (z2 - z1)^2

    vaddsd xmm3, xmm0, xmm1    ; suma parcial
    vaddsd xmm4, xmm3, xmm2    ; suma de cuadrados
    vsqrtsd xmm0, xmm0, xmm4   ; raíz cuadrada
    vmovsd Distancia, xmm0

    call ExitProcess
main ENDP
```

Puntos clave del código:

- qword reserva 8 bytes, adecuado para un double de 64 bits.
- vmovsd mueve un escalar double entre memoria y registros XMM.
- vsubsd, vmulsd, vaddsd y vsqrtsd operan sobre escalares double.
- El prefijo v indica la variante AVX/VEX, que suele usar sintaxis de tres operandos: destino, fuente1, fuente2.
- Visual Studio permite ver registros YMM/XMM y memoria para confirmar los valores en hexadecimal y como double.

5. Comparaciones de punto flotante

A diferencia de la aritmetica x87 antigua, con AVX/SSE el resultado de una comparacion de flotantes puede reflejarse en RFLAGS. El detalle importante es que en punto flotante existen NaN e infinitos, por lo que una comparacion no siempre produce solo menor, mayor o igual: tambien existe el caso unordered o incomparable.

Condicion	ZF	PF	CF
XMM0 > XMM1	0	0	0
XMM0 = XMM1	1	0	0
XMM0 < XMM1	0	0	1
Incomparables / unordered	1	1	1

La instruccion vcomis[s|d] funciona de forma parecida a cmp: compara operandos flotantes y actualiza banderas. La variante s es para precision simple y la variante d para double.

Tambien se presento la familia vcmps[s|d], que no solo actualiza banderas, sino que genera una mascara de bits en el destino. Por ejemplo:

```
vcmps d xmm2, xmm0, xmm1, CMP_EQ
```

Esta instruccion compara los 64 bits bajos de xmm0 y xmm1. Si son iguales, los 64 bits bajos de xmm2 se ponen en 1s; si no son iguales, se ponen en 0s. CMP_EQ corresponde a la constante 0x00.

6. AVX empaquetado: SIMD y operaciones horizontales

La idea de AVX empaquetado es realizar la misma operacion sobre varios datos en paralelo dentro de un mismo registro. Esto es SIMD: Single Instruction, Multiple Data.

En operaciones normales empaquetadas, cada elemento se combina con el elemento en la misma posicion del otro registro: X0 con Y0, X1 con Y1, X2 con Y2, etc. Esto sirve para vectores, imagenes, audio, matrices y calculos cientificos.

AVX tambien incluye operaciones horizontales, donde se operan elementos adyacentes dentro de los registros. El ejemplo visto fue:

```
vhaddps xmm3, xmm1, xmm2
```

Si xmm1 contiene [0.1, 0.2, 0.3, 0.4] y xmm2 contiene [0.5, 0.6, 0.7, 0.8], el resultado mostrado fue [0.3, 1.1, 0.7, 1.5], porque suma pares adyacentes.

7. Cosas que probablemente sirven para el trabajo

- Distinguir overflow wraparound de saturacion. En wraparound el valor da la vuelta; en saturacion se pega al maximo/minimo.
- Recordar que AVX usa YMM0-YMM15 de 256 bits, y que los XMM son la mitad baja de esos registros.
- MXCSR es el registro de control/status para operaciones FP SIMD/AVX: banderas, excepciones y redondeo.
- Para double escalar se usan instrucciones con sufijo sd: vmovsd, vsubsd, vmulsd, vaddsd, vsqrtsd.
- Para floats empaquetados se usan sufijos como ps; para doubles empaquetados, pd.
- Las comparaciones flotantes tienen caso incomparable por NaN. Por eso aparece PF junto con ZF y CF.
- En operaciones empaquetadas normales se opera por posicion. En operaciones horizontales se operan elementos adyacentes.

8. Mini-glosario rapido

Termino	Significado
Wraparound	Overflow permitido; resultado modular dentro del rango.
Saturacion	Limita el resultado al maximo o minimo representable.
XMM	Registro de 128 bits; usado por SSE y por escalares FP.
YMM	Registro de 256 bits; usado por AVX.
MXCSR	Registro de control/status para FP SIMD.
sd	Scalar double: operacion escalar de 64 bits.
ss	Scalar single: operacion escalar de 32 bits.
ps	Packed single: floats empaquetados.
pd	Packed double: doubles empaquetados.
NaN / unordered	Comparacion flotante incomparable.

Resumen preparado a partir del video de clase mayo-5.mp4. Enfocado en repaso rapido y apoyo para trabajo practico.